

WHAT THIS SESSION IS

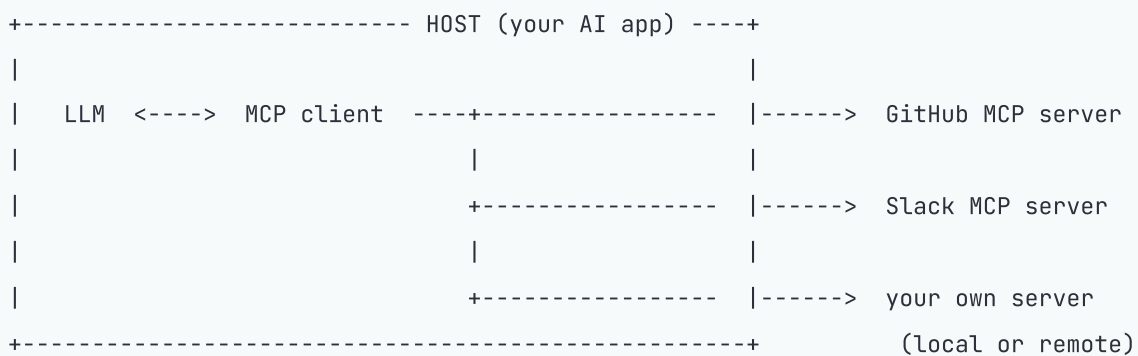
Run on 13 September 2026, the day after session 7, with Akshay Agarwal continuing MCP. The first two and a half hours stay on the protocol: what a JSON-RPC message looks like, the real payloads for tools, resources and prompts, why Anthropic built MCP on JSON-RPC instead of plain REST, the two transports, and the three phases every connection goes through. The last hour is code. We build an MCP server with FastMCP that exposes two tools over stdio, then a LangChain client that starts that server, loads its tools, and hands them to a model. The code fought back in the live class, and those errors are written up here too, because each one teaches something you will run into yourself.

CONTENTS

1. Where this session picks up
2. JSON-RPC, word by word
3. Request, result, error: the three shapes
4. Real payloads: listing and calling tools
5. Resources: list, read, subscribe, unsubscribe
6. Notifications: the message with no id
7. Prompts: list first, then get
8. Batching
9. Why MCP is built on JSON-RPC
10. Transports: stdio and Streamable HTTP
11. The lifecycle in three phases
12. Initialization, step by step
13. Client capabilities: roots, sampling, elicitation
14. Server capabilities
15. Operation and shutdown
16. A heads-up about the newest spec revision
17. Setting up the project
18. The server: my_mcp_server.py
19. The client: my_mcp_client.py
20. What broke in class, and what it teaches
21. A cleaner version to try yourself
22. Clarifications from the Q&A

1 Where this session picks up

A quick recap of yesterday before anything new. An MCP setup has three parts. The **host** is your AI application. Inside it you create an **MCP client**, and that client talks to one or more **MCP servers**. A server might belong to GitHub, Slack or X, or it might be one you wrote yourself and run on your own laptop. You only need your own server when you want to add something of your own. Otherwise your client connects straight to the servers other people publish.



A server can expose three things, the primitives:

Primitive	What it is	Example
Tools	Things the server can <i>do</i> . Functions.	search tweets, create a PR
Resources	Data the server can <i>provide</i> .	a user profile, a README, documentation
Prompts	Ready-made prompt templates written by the people who know the domain.	"review this pull request"

A CORRECTION TO YESTERDAY: RESOURCES ARE NOT ONLY STATIC

Yesterday resources were described as static content. The instructor corrected that today. Call them **data**. A README barely changes, but a user profile or a list of tweets changes all the time. That is exactly why resources have `subscribe` and `unsubscribe` : so a client can be told when the data behind a resource changes.

Last time also ended on the **data layer**: the rules for how client and server exchange messages. MCP's data layer is JSON-RPC, and the question left open was why Anthropic chose it over ordinary HTTP APIs. That is

where today starts.

2 JSON-RPC, word by word

- **JSON** is JavaScript Object Notation, the text format you already know from APIs.
- **RPC** is Remote Procedure Call. *Remote* means the function you want is not on your machine. *Procedure* just means a function. So an RPC is calling a function that lives on some other machine or process, as if it were local.

Put together: JSON-RPC is a small set of rules for calling a remote function where every message is written in JSON. JSON gives the body its structure. RPC is the idea of what that body is for, which is naming a function, passing it arguments, and getting a result back.

JSON AND JSON-RPC ARE NOT THE SAME THING

A student mixed these up in class. JSON is only a format for writing data. JSON-RPC is a protocol that says which keys a message must have (`jsonrpc` , `method` , `params` , `id`) and what each one means. Every JSON-RPC message is JSON, but most JSON is not JSON-RPC. It is also language independent: Python, Java, Rust and JavaScript all have libraries for it, because at the end of the day it is just text.

3 Request, result, error: the three shapes

Here is the smallest possible request, using the `add` example from yesterday:

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [4, 2],
  "id": 1
}
```

- `jsonrpc` is the version of JSON-RPC. It is always the string `"2.0"` .
- `method` is the name of the function you want to run on the other side.
- `params` are its arguments. They can be a list, as here, or an object with named keys, which is what MCP uses.
- `id` ties the answer back to this question.

If it works, the response carries the same `id` and a `result` . There is no `method` or `params` in a response:

```
{
  "jsonrpc": "2.0",
  "result": 6,
  "id": 1
}
```

If it fails, say the server has no `add` method, you get an `error` object instead of a `result`. Never both.

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32601,
    "message": "Method not found"
  },
  "id": 1
}
```

CORRECTION: THERE IS NO JSON-RPC 3.0

In class it was said that the current version might be 3.0. It is not. The only version in use is **2.0**, published in 2010, and MCP requires `"jsonrpc": "2.0"` in every message. Do not put any other value there.

Error codes look strange next to HTTP's 404 and 500, but there is a short fixed list, and you do not need to memorise it. These are the standard ones from the JSON-RPC 2.0 specification:

Code	Meaning	When you see it
-32700	Parse error	The message was not valid JSON at all
-32600	Invalid Request	Valid JSON, but not a valid JSON-RPC request
-32601	Method not found	The method name does not exist on the server
-32602	Invalid params	Wrong or missing arguments
-32603	Internal error	Something broke inside the server

An error can also carry an optional `data` field with extra detail.

Why the id matters

Many requests can be in flight at once, and answers do not have to come back in order. The `id` is the only thing that says which answer belongs to which question. If you sent a request with id 1 and get a response with id 2, that response is for some other request. MCP adds two rules on top: an id must be a string or a number (never `null`), and the same id must not be reused for another request in the same session. You will not manage ids by hand, though. The SDK does it for you. You only need to understand them so the messages make sense when you read them in a log.

CLIENT	SERVER
request id=1 method=tools/list	
----->	
request id=2 method=resources/list	
----->	
response id=2 result=[...resources...]	id=2 finished first
<-----	
response id=1 result=[...tools...]	
<-----	

the client matches each answer to its question by id, not by order

4 Real payloads: listing and calling tools

Yesterday's list of standard methods (`tools/list` , `tools/call` , `resources/list` and so on) was just names. Today each one got a real message. The examples used an imaginary X (Twitter) server. They were illustrations prepared for the class, not copies of a real server's output, so do not read too much into individual fields.

tools/list: what can you do?

Before a client can use anything, it has to ask what exists. That is `tools/list` .

```
// request
{ "jsonrpc": "2.0", "id": 1, "method": "tools/list" }

// response
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "search_tweets",
        "description": "Search recent tweets on X",
        "inputSchema": {
          "type": "object",
          "properties": {
            "query": { "type": "string" },
            "limit": { "type": "integer" }
          },
          "required": ["query"]
        }
      },
      {
        "name": "get_user",
        "description": "Get information about an X user",
        "inputSchema": {
          "type": "object",
          "properties": { "username": { "type": "string" } }
        }
      }
    ]
  }
}
```

Every tool comes with three things, and they are the same three you gave LangChain tools in sessions 4 and 5: a **name**, a **description**, and an **input schema**. The properties inside `inputSchema` become the parameters of the call. To make it concrete, the instructor rewrote the entry for `add` :

```
{
  "name": "add",
  "description": "Add two numbers",
  "inputSchema": {
    "type": "object",
    "properties": {
      "num1": { "type": "integer" },
      "num2": { "type": "integer" }
    },
    "required": ["num1", "num2"]
  }
}
```

THIS IS THE TOOL DEFINITION THE MODEL SEES

Think back to `bind_tools`. The model decides whether to call a tool by reading its name, description and schema. MCP simply moves those three things to the server, so whoever builds the tool also writes its description. It is also why a vague description on an MCP server hurts you exactly the way a vague docstring did in session 4.

tools/call: run one

```
// request
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "search_tweets",
    "arguments": { "query": "mcp", "limit": 5 }
  }
}

// response
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      { "type": "text", "text": "Tweet 1: MCP is changing how agents use tools..." },
      { "type": "text", "text": "Tweet 2: Built my first MCP server today..." }
    ],
    "isError": false
  }
}
```

`params` holds the tool's `name` and its `arguments`. The result is a `content` list, and each item has a `type` such as `text` or `image`. Remember this shape. It is exactly why `result[0]['text']` shows up in the client code at the end of the session.

`isError` is worth knowing about. If the tool ran but failed (the X API was down, say), the server still sends a normal `result` with `isError: true` and an explanation in `content`, so the model can read what went wrong. A JSON-RPC `error` is kept for protocol problems, like an unknown tool name.

5 Resources: list, read, subscribe, unsubscribe

If tools are what a server can do, resources are the data it can give you.

resources/list

```
// request
{ "jsonrpc": "2.0", "id": 3, "method": "resources/list" }

// response
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "resources": [
      { "uri": "x://users/elonmusk/profile",
        "name": "Elon Musk profile",    "mimeType": "application/json" },
      { "uri": "x://users/elonmusk/tweets",
        "name": "Elon Musk tweets",    "mimeType": "application/json" },
      { "uri": "x://users/elonmusk/bookmarks",
        "name": "Elon Musk bookmarks", "mimeType": "application/json" }
    ]
  }
}
```

Each resource has a **URI** (its address), a **name**, and a **MIME type**, which says what kind of content it is: `text/plain`, `application/json`, `image/png` and so on.

resources/read

```
// request
{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "resources/read",
  "params": { "uri": "x://users/elonmusk/tweets" }
}

// response
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {
    "contents": [
      {
        "uri": "x://users/elonmusk/tweets",
        "mimeType": "application/json",
        "text": "[ ...tweets... ]"
      }
    ]
  }
}
```

You pick a URI you got from the list and read it. Want bookmarks instead of tweets? Change the URI. Nothing else changes.

resources/subscribe and the update that follows

Say you want to know every time new tweets appear. You subscribe to that URI:

```
// request
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "resources/subscribe",
  "params": { "uri": "x://users/elonmusk/tweets" }
}

// response: an empty result simply means "subscribed"
{ "jsonrpc": "2.0", "id": 5, "result": {} }
```

Later, when that data changes, the server sends this on its own, without anyone asking:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": { "uri": "x://users/elonmusk/tweets" }
}
```

Note that the notification only says *which* resource changed. It does not carry the new data. The client calls `resources/read` again when it wants the fresh copy.

resources/unsubscribe

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "method": "resources/unsubscribe",
  "params": { "uri": "x://users/elonmusk/tweets" }
}
```

Again an empty result comes back as confirmation, and from then on there are no more update notifications for that URI.

CLIENT	SERVER
resources/subscribe id=5 uri=.../tweets	
----->	
result {} id=5	
<-----	
... time passes, new tweet ...	
notifications/resources/updated (no id)	
<-----	server starts it
resources/read id=7 uri=.../tweets	
----->	fresh copy
result contents=[...] id=7	
<-----	

6 Notifications: the message with no id

Look again at the update notification. Every other message so far had an `id`. This one does not, and that is deliberate.

An id exists so a response can be matched to its request. A notification is not a response to anything, and it does not expect anything back. It is fire and forget. Nobody has to reply to it, so there is nothing to match, and no id. In JSON-RPC terms, that is the whole definition: a message with a `method` but no `id` is a notification, and the other side must not answer it.

	Request	Response	Notification
Has <code>method</code>	Yes	No	Yes
Has <code>id</code>	Yes	Yes, same as the request	No
Expects a reply	Yes	No	No
Example	<code>tools/call</code>	<code>result</code> or <code>error</code>	<code>notifications/resources/updated</code>

Can a plain REST API do this? Not by itself. With REST the client always asks first. To push something from the server you add extra machinery such as Server-Sent Events or WebSockets. JSON-RPC has notifications built into the message format. Notifications mostly flow from server to client, but a client can send them too, and you will see one in the initialization steps below.

7 Prompts: list first, then get

```
// request
{ "jsonrpc": "2.0", "id": 8, "method": "prompts/list" }

// response
{
  "jsonrpc": "2.0",
  "id": 8,
  "result": {
    "prompts": [
      {
        "name": "analyze_tweet",
        "description": "Analyse the sentiment of a tweet",
        "arguments": [ { "name": "tweet_url", "required": true } ]
      },
      {
        "name": "create_tweet_thread",
        "description": "Write a tweet thread on a topic",
        "arguments": [ { "name": "topic", "required": true } ]
      }
    ]
  }
}
```

Then, with a name and arguments taken from that list, the client asks for one prompt:

```
// request
{
  "jsonrpc": "2.0",
  "id": 9,
  "method": "prompts/get",
  "params": {
    "name": "analyze_tweet",
    "arguments": { "tweet_url": "https://x.com/..." }
  }
}

// response
{
  "jsonrpc": "2.0",
  "id": 9,
  "result": {
    "description": "Analyse the sentiment of a tweet",
    "messages": [
      {
        "role": "user",
        "content": { "type": "text", "text": "Analyse this tweet: ..." }
      }
    ]
  }
}
```

The point made in class applies to all three primitives: **the client knows nothing about a server until it lists**. Before `prompts/list`, it has no idea a prompt called `analyze_tweet` exists or what arguments it takes. So the order is always list first, then get, read or call. The list also carries the arguments each item needs, so the client can fill them in correctly.

Prompts come back as ready-made messages. Remember the pull request example from yesterday: a good server prompt already knows which details a PR review needs, so you do not end up pasting IDs, branch names or credentials into a prompt of your own.

8 Batching

JSON-RPC 2.0 lets you send several requests in a single message by putting them in a list. Each keeps its own id, and the answers come back as a list too:

```
// one message, two requests
[
  { "jsonrpc": "2.0", "id": 1, "method": "tools/call",
    "params": { "name": "search_tweets", "arguments": { "query": "mcp" } } },
  { "jsonrpc": "2.0", "id": 2, "method": "tools/call",
    "params": { "name": "get_user", "arguments": { "username": "telusko" } } }
]

// one message, two answers, matched by id
[
  { "jsonrpc": "2.0", "id": 1, "result": { "content": [ ... ] } },
  { "jsonrpc": "2.0", "id": 2, "result": { "content": [ ... ] } }
]
```

The advice from class is sound for any kind of parallel calls: only group requests that do not depend on each other. If call B needs the output of call A, send them one after the other. If one call is slow, grouping it with fast ones just makes the fast ones wait.

WORTH KNOWING: MCP ITSELF NO LONGER USES BATCHING

Batching is a real JSON-RPC 2.0 feature and it was taught as one of its advantages. MCP's own history with it is short, though. Batch support was added in the 2025-03-26 revision of the MCP spec and removed again in 2025-06-18. In current revisions each HTTP POST carries exactly one JSON-RPC message. Understand batching as part of JSON-RPC, but do not expect to send batches to an MCP server. If you need parallel calls, the SDK sends separate requests concurrently, and the ids keep them apart.

9 Why MCP is built on JSON-RPC

This was the open question from yesterday. REST APIs work fine for most apps, so why did Anthropic pick JSON-RPC for MCP? The class listed these reasons.

Reason	What it means for MCP
Lightweight	Plain JSON with only four or five keys, nothing heavy on top.
Bidirectional	In REST the client always starts and the server only answers. In JSON-RPC either side can send a message. MCP needs this, because servers sometimes have to reach out to the client.
Transport agnostic	REST is tied to HTTP. JSON-RPC does not care how the text travels: HTTP, the standard input and output of a process, a socket, or something you invent. That is how the same MCP messages work for a server on your laptop and one on the internet.
Notifications built in	Fire and forget messages with no id, with no extra machinery needed.
Batching	Part of JSON-RPC 2.0 (see the note in section 8 about MCP).
Language independent	It is only JSON text, so servers and clients can be written in any language.

REST	JSON-RPC (as used by MCP)
client ---- request ----> server client <--- response ---- server	client <---- messages ----> server either side may start
only over HTTP server never speaks first	over stdio, HTTP, sockets, custom... notifications need no reply

A NOTE ON THE SOCKET EXAMPLE

To show that there are ways of moving data besides HTTP, the class used chat apps like WhatsApp, which use sockets: a connection that stays open so both sides can send messages whenever they like. That is a good way to picture a transport, but sockets are not one of MCP's official transports. The spec defines two, stdio and Streamable HTTP, and allows custom transports on top of those as long as they carry the same JSON-RPC messages.

10

Transports: stdio and Streamable HTTP

JSON-RPC decides what a message looks like. The **transport** decides how it gets from one side to the other. MCP has two standard ones, and the choice mostly comes down to where the server runs.

LOCAL SERVER (same machine)

REMOTE SERVER (another machine)

```
+----- your laptop -----+      +--- laptop ---+      +--- cloud ---+
|                               |      |               | HTTP |               |
| client ---- stdin ----> server |      | client ---|----->|-- server  |
| client <--- stdout ---- process |      |               <--|-----|--          |
|                               |      |               | POST + |               |
| no port, no network           |      |               | SSE   | /mcp        |
+-----+                      +-----+      +-----+

transport="stdio"                transport="http" (Streamable HTTP)
```

stdio

stdio stands for standard input and output. Every program already has these two streams: `stdin`, which it reads from, and `stdout`, which it writes to. If you have written C or C++, it is the same `stdin` and `stdout`. With stdio, the client starts the server as a **child process** and writes JSON-RPC messages into the server's stdin, one per line. The server writes its replies to its stdout.

- **Local only.** Client and server are on the same machine.
- **Faster.** Asked in class which is faster locally, most people guessed right: stdio. Nothing goes over the network and no port has to be opened.
- **More secure.** Nothing is exposed to the network, so nobody else can connect.
- **Simple.** There is no web server to set up.

Streamable HTTP

For a remote server, stdio is not an option, because you cannot write into the stdin of a process on another machine. So remote servers use HTTP. The server runs on its own as a normal web service with a single endpoint, usually `/mcp`. The client sends every JSON-RPC message as an **HTTP POST** to that endpoint. The server can answer with a single JSON response, or it can open a **Server-Sent Events (SSE)** stream and send several messages back over time.

SSE is a web standard on top of HTTP that lets a server keep a response open and push events down it. The class compared it to calling `llm.stream()` instead of `llm.invoke()` back in session 4: instead of waiting for one big JSON at the end, you see things arrive chunk by chunk. For MCP this matters with long-running tools. A tool that checks thousands of database rows can send progress and log messages while it works, so the client is not left staring at nothing.

"BUT THE LLM RUNS ON MY SIDE, SO WHY STREAM?"

A good question from class. Streaming here is not about the model's tokens. It is about the *tool*. When the model calls a slow tool on a remote server, you still want to see what that tool is doing, not only its final answer. Do not confuse it with `agent.stream()` either. It is the same idea, but at a different layer.

	stdio	Streamable HTTP
Where the server runs	Same machine	Anywhere reachable over HTTP
Who starts the server	The client launches it	You run it yourself, separately
Port needed	No	Yes (for example 8000)
Speed	Fastest	Network latency applies
Clients per server	One (its parent)	Many
Use it for	Local tools, personal projects, development	Shared servers, SaaS integrations

A local server can offer both. Start with stdio while you build it, and add HTTP later if you decide to share it.

ABOUT "HTTP WITH SSE"

You will find older blog posts and servers that talk about an "HTTP + SSE" transport with two separate endpoints. That was MCP's first remote transport, from the 2024-11-05 revision. It has been replaced by Streamable HTTP, which uses one endpoint and treats SSE as an optional way to stream. If a tutorial sets up two endpoints, it is out of date. The instructor made the same point about videos in general: MCP is moving fast, so anything more than a month or two old should be checked against the official documentation.

11 The lifecycle in three phases

A client does not simply connect and start calling tools. Every connection goes through three phases, and the instructor walked through them using the lifecycle page of the official spec.

CLIENT

SERVER

```
===== 1. INITIALIZATION =====
| initialize (protocolVersion, capabilities, clientInfo)|
|----->|
| result (protocolVersion, capabilities, serverInfo) |
|<-----|
| notifications/initialized (no id) |
|----->|

===== 2. OPERATION =====
| tools/list, tools/call, resources/read, prompts/get ...|
|<----->|

===== 3. SHUTDOWN =====
| close the stream (stdio) / close the connection (HTTP) |
|----->|
connection closed
```

The analogy used in class: phase 1 is the interview and the offer, phase 2 is the actual job, and phase 3 is leaving. The next sections take them one at a time.

12 Initialization, step by step

Initialization sets the rules for the whole connection. It must be the very first thing that happens, and it always takes three steps.

Step 1: the client says hello

The **client always starts**. It sends an `initialize` request with three pieces of information:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "roots": { "listChanged": true },
      "sampling": {},
      "elicitation": {}
    },
    "clientInfo": {
      "name": "ExampleClient",
      "title": "Example Client Display Name",
      "version": "1.0.0"
    }
  }
}

```

1. **protocolVersion.** Which revision of MCP the client speaks. It is written as a date, because MCP is versioned by the date a revision was published.
2. **capabilities.** What the client can offer the server (section 13).
3. **clientInfo.** Who the client is: a name, a display title, a version.

Step 2: the server answers with its own details

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "logging": {},
      "prompts": { "listChanged": true },
      "resources": { "subscribe": true, "listChanged": true },
      "tools": { "listChanged": true }
    },
    "serverInfo": {
      "name": "ExampleServer",
      "title": "Example Server Display Name",
      "version": "1.0.0"
    }
  }
}
```

Same three parts, from the other side: the protocol version it will use, its capabilities, and `serverInfo`. The server *must* answer an `initialize`. There is no option to ignore it.

CAPABILITIES ARE NOT THE LISTS

Look closely at the server's capabilities. It says it *has* tools, resources and prompts, but it does not list them. No tool names, no URLs. This is only an introduction. The actual lists come later, in the operation phase, when the client asks with `tools/list` and the rest. The sub-flags matter too:

`listChanged: true` means the server will send a notification when that list changes, and `subscribe: true` on resources means you can subscribe to individual resources.

Step 3: the client confirms

The client now looks at what the server sent back. If the version and capabilities work for it, it sends one last message:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

No id, because it is a notification. The client is not asking a question, only confirming. After this, the connection is established and normal work can begin.

IF THE CLIENT DOES NOT LIKE THE ANSWER, THERE IS NO STEP 3

Say the client asked for one protocol version and the server replied with another that the client does not support. The client simply disconnects. It does not send a polite "no thanks" message. The server finds out only because `notifications/initialized` never arrives. This is the most important step of the three, because only after it can you say a connection exists.

Version negotiation

The rules, as the spec states them:

- The client sends a version it supports. This *should* be the latest one it supports.
- If the server supports that version, it *must* reply with the same version.
- If not, it replies with another version it supports, ideally its latest.
- If the client does not support the version the server replied with, it *should* disconnect.

A client can support several versions at once, so it checks the server's reply against its own list, not against a single value.

Capability negotiation

Each side learns what the other can do, and from then on both are held to it. If the client said it supports sampling, it has to handle sampling requests when they come. If the server said it supports resource subscriptions, it has to deliver update notifications. The class compared it to a job offer: the company promises the salary and benefits, the employee promises their skills and time, and neither side can back out of those promises later. During operation, both sides may only use features that were agreed during initialization.

WHY SEND STEP 3 WHEN THE SERVER ALREADY REPLIED?

A student asked this after the break. In step 1 the client only describes itself, and the server does not check anything. In step 2 the server only describes itself. Nobody has actually said yes until step 3. It is the acknowledgement that closes the deal, and it is part of the standard, so every client sends it.

ABOUT THE VERSION DATES SHOWN IN CLASS

The examples in class used 2025 and 2026 dates just to show a mismatch, and the instructor was clear that you should not copy them. You also never type this value yourself. The SDK fills in the protocol version it supports and does the whole handshake for you, as you will see in the code. If you want to know the current revision, check the specification page linked at the end.

13 Client capabilities: roots, sampling, elicitation

In MCP the server is not only a passive tool provider. Sometimes it needs something from the client. Client capabilities are the things a client says it is willing to give.

roots: which folders the server should work in

Roots tell the server which directories or files it should work within, for example the project folder you have open. Think of Claude Code or Cursor asking whether you trust a folder before they work in it. Roots let the client say "this is the folder we are working in", so the server does not go wandering over your whole disk.

One nuance on top of what was said in class. Roots are information about boundaries, not a permission system. They do not grant read, edit or delete rights by themselves. What a server can actually touch is still decided by the operating system and by the client application, which should ask you before doing anything risky.

sampling: the server may borrow the client's LLM

Your client app already has a model. That is the whole reason you are loading tools into it. Sampling means the client says: *server, if you ever need an LLM while doing your work, you may ask me to run a prompt through mine.*

CLIENT (has the LLM)	SERVER (has no LLM)
tools/call summarize_pr	
----->	
	needs an LLM to
sampling/createMessage "summarise this diff"	write the summary
<-----	
runs it on the client's model	
(a well built client asks the user first)	
result: "This PR adds..."	
----->	
tools/call result	
<-----	

The server does not need its own model or API key, and the person using the client stays in control of which model runs and what gets sent. In class the instructor said the server could ask for "my LLM or maybe anything else". Strictly speaking, sampling in the spec is only about LLM completions. The request is called `sampling/createMessage`.

elicitation: the server may ask the user for missing details

Without elicitation, a request that is missing information simply fails with an error. With it, the server can stop and ask the user for what is missing, and the client shows that question. The class used a pull request review

example: you ask the server to review a PR but forget to say which PR or which branch. Instead of failing, the server asks: "Which pull request number?"

	Sampling	Elicitation
Server wants	An LLM completion	Information from the user
Triggered by	The server needing a model to do its job	Something missing or unclear in the request
Answered by	The client's model	The human, through the client's UI
Request name	sampling/createMessage	elicitation/create

CORRECTION: NEVER USE ELICITATION FOR API KEYS OR PASSWORDS

One of the examples in class was a server asking for a missing API key through elicitation. The spec forbids exactly that: servers **must not** use form-mode elicitation to ask for passwords, API keys, access tokens or payment details. Anything typed into a form passes through the client, and sensitive data should not. For secrets, the 2025-11-25 revision added a separate URL mode, where the user is sent to a secure web page and the secret never goes through the client at all. The branch name or PR number version of the example is the correct one.

Both sampling and elicitation are permissions. If the client does not declare them, the server cannot use them. As said in class, the server cannot do anything on its own that the client has not allowed.

14 Server capabilities

Capability	What the server is saying
tools	I have tools you can list and call.
resources	I have resources you can list and read, and maybe subscribe to.
prompts	I have prompt templates you can list and get.
logging	I can send you structured log messages while I work.

In class, logging was explained next to streaming, because log messages are one of the things that flow back while a long task runs. The two are separate features, though. A server sends a log as a `notifications/message`, and it can do that on any transport.

The spec's capability table also lists `completions` (autocomplete for arguments) and `experimental`, plus a few newer entries. The instructor left these out deliberately and asked that nobody bring them up yet. They

will come later when they are needed.

15 Operation and shutdown

Phase 2: operation

Now the actual work happens. The client lists what the server offers and uses it: calls tools, reads resources, gets prompts. The server can use whatever the client agreed to, like sampling or elicitation. The only rules are that both sides keep to the agreed protocol version and only use capabilities that were negotiated.

```
initialization told us: "this server has tools"
                        |
                        v
operation:  tools/list ---> [ greet, add ] ---> model picks "add"
                                                |
                                                v
                        tools/call add {num1: 4, num2: 5} ---> content: "9"
```

Phase 3: shutdown

Usually the client ends things, because it started them. The key point, and the instructor said it was the only line here that really matters: **there is no JSON-RPC message for shutting down**. The transport does it.

- **stdio**: the client closes the server's input stream. If the server does not exit within a reasonable time, the client sends it `SIGTERM`, and if it still does not exit, `SIGKILL`.
- **HTTP**: the client closes the HTTP connection.

If the client wants that server again later, it starts over with a new initialization.

When the server is the one that goes away

A server can also crash or be shut down without warning, and it will not tell anyone. The client has to cope with that. This is no different from any other backend: with a REST API you get a 500 or a connection error, and you have logging, alerts and maintenance notices. The same habits apply to MCP.

- **Timeouts**. The spec says implementations *should* set a timeout on every request. If there is no answer in time, the client sends a cancellation notification and stops waiting. Then it can retry, or tell the user the feature is not available right now.
- **Ping**. Either side can send a `ping` request just to check that the other one is still alive.
- **Progress**. A long task can send progress notifications, and a client may reset its timeout when they arrive, while still enforcing a hard maximum.

Whether a connection closes after a period of inactivity is up to the implementation, just like login sessions on a website. Some keep you logged in until tomorrow, others log you out after a few minutes.

16 A heads-up about the newest spec revision

READ THIS BEFORE YOU GO TO THE SPEC WEBSITE

Everything in sections 11 to 15 follows the lifecycle as it was taught, which is the model defined by the **2025-11-25** revision and earlier. It is still how a very large number of servers and SDKs work, and every idea in it (versions, capabilities, identity, listing before calling) carries over.

But the specification moves quickly. The newest revision, **2026-07-28**, makes MCP stateless. When you open the spec site you may see these changes, so do not be surprised:

- The `initialize` / `notifications/initialized` handshake is removed. Each request carries its own protocol version and client capabilities in a `_meta` field, and a new `server/discover` method lets a client ask up front what a server supports. The spec calls the handshake-based revisions "legacy" and lets servers support both.
- `ping` is removed, and `resources/subscribe` / `unsubscribe` are replaced by a single `subscriptions/listen` stream.
- Servers no longer send requests to clients directly. Sampling and elicitation happen by the server returning an "input required" result, and the client retries the original request with the answers.
- Roots, sampling and logging are marked deprecated (still working, but not recommended for new code). The suggested replacements: pass folders as tool arguments, call LLM providers directly, and log to stderr.

For this course, follow what the instructor builds in class. The SDK handles whichever revision it speaks, so your Python code does not change. Just be aware of which revision a page describes when you read the documentation: the version is always in the URL.

17 Setting up the project

After two days of theory, the class moved to code with a new project built from scratch, which doubles as a revision of the uv workflow from session 2.

```
uv init                # creates pyproject.toml, main.py, .python-version
uv sync                # creates .venv and installs what pyproject lists

uv add fastmcp          # to build the MCP server
uv add openai langchain langchain-openai python-dotenv
uv add langchain-mcp-adapters mcp # to load MCP tools into LangChain
```

`uv venv` would also create the virtual environment. `uv sync` does that and installs the dependencies in one go.

Then a `.env` file with an OpenAI key, created on the OpenAI platform with a 30-day expiry:

```
OPENAI_API_KEY=sk-...
```

Only the client needs this key. The server has no model in it. The project ends up with two files: `main.py` renamed to `my_mcp_server.py`, and a new `my_mcp_client.py`. The final `pyproject.toml` in the shared code:

```
dependencies = [
    "fastmcp>=3.4.7,<4.0.0",
    "langchain>=1.4.0",
    "langchain-mcp-adapters>=0.3.1",
    "langchain-openai>=1.6.2",
    "mcp>=1.30.0",
    "openai>=3.13.0",
    "python-dotenv>=1.2.3",
]
```

WHICH PACKAGE DOES WHAT

- **fastmcp**: a high-level framework for writing MCP servers. It turns ordinary Python functions into MCP tools and handles the protocol for you.
- **mcp**: the official Python SDK, the lower-level protocol implementation. `fastmcp` and `langchain-mcp-adapters` both build on it.
- **langchain-mcp-adapters**: converts MCP tools into LangChain tools, so they work with `bind_tools` and `create_agent`.

18 The server: `my_mcp_server.py`

This is the file as shared after class:

```

from fastmcp import FastMCP

mcp_server = FastMCP("FirstMCPServer")

@mcp_server.tool
def greet():
    '''This is the tool to greet users'''
    return "Hello from Telusko, we hope you are fine?"

@mcp_server.tool
def add(num1, num2):
    '''This is the tool to add two numbers and give the output'''
    return 20

def main():
    mcp_server.run(transport="stdio")

if __name__ == "__main__":
    main()

```

Line by line

- `FastMCP("FirstMCPServer")` creates the server. The name is what the server reports as `serverInfo` during initialization, so clients know who they are talking to.
- `@mcp_server.tool` registers a function as a tool on *this* server. The first version in class used a bare `@tool`, which is LangChain's decorator. That was fixed right away: an MCP tool has to be attached to the server object. You do not need LangChain on the server side at all.
- **The function name** becomes the tool name (`greet` , `add`).
- **The docstring** becomes the tool description. This is not a comment for humans. It is what `tools/list` sends to the client, and what the model reads to decide whether to call the tool.
- **The parameters** become the input schema. `greet` has none. `add` has `num1` and `num2` .
- `mcp_server.run(transport="stdio")` starts the server and waits for JSON-RPC messages on stdin.
- `if __name__ == "__main__":` is ordinary Python, asked about in class. It runs `main()` only when the file is executed directly, not when it is imported.

WHY ADD RETURNS 20

That hard-coded `return 20` is not a mistake left in by accident. It was a debugging trick from the end of the class. The model kept giving correct answers, and a `print` inside the tool showed nothing, so there was no way to tell whether the tool was really being called. Returning 20 for every sum proved it (section 20). The real version is `return num1 + num2`. Also add type hints: without them FastMCP produces a schema where `num1` and `num2` have no type at all, and the model has to guess what to send.

Running it directly with `uv run my_mcp_server.py` shows the FastMCP banner and "Starting MCP server 'FirstMCPServer' with transport 'stdio'". Then it just sits there waiting for input. That is fine, but as the class found out a few minutes later, you do not need to do this for stdio.

19 The client: my_mcp_client.py

The client was built up in stages. First, a plain chat call with no MCP at all, to make sure the model worked. Asked "hi", it answered "Hello! How can I assist you today?", which is a generic reply and not our greeting, because no tools were loaded yet. Then the MCP parts were added. This is the final file as shared:

```

from dotenv import load_dotenv
import sys
import asyncio
from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage
from langchain_openai import ChatOpenAI
from pathlib import Path
from langchain_mcp_adapters.client import MultiServerMCPClient

load_dotenv()

server_params = {
    "FirstMCPServer": {
        "command": sys.executable,
        "args": [str(Path(__file__).parent / "my_mcp_server.py")],
        "transport": "stdio",
    }
}

async def main():
    mcp_server = MultiServerMCPClient(server_params)
    tools = await mcp_server.get_tools()
    print(tools)

    llm = ChatOpenAI(model="gpt-4o-mini").bind_tools(tools)

    history = [HumanMessage(content="Add 4 and 5")]
    response = llm.invoke(history)
    history.append(response)

    toolcall = response.tool_calls[0]
    tool = next(t for t in tools if t.name == toolcall["name"])
    result = await tool.ainvoke(toolcall["args"])
    print(result[0]['text'])

    history.append(ToolMessage(content=result[0]['text'], tool_call_id=toolcall["id"]))
    response = await llm.ainvoke(history)
    print(response.content)

```

```
if __name__ == "__main__":
    asyncio.run(main())
```

The server configuration

- The key, `"FirstMCPServer"`, is a label you choose for this connection. It is called "multi server" because you can list several servers in this one dictionary.
- `"command": sys.executable` is the full path of the Python interpreter running the client right now, meaning the one inside your `.venv`. Using it guarantees the server starts with the same environment, so `fastmcp` is found.
- `"args"` is the path to the server file. `Path(__file__).parent` is the folder this client file is in, so it works no matter which directory you run from. The instructor's advice was not to memorise this line. It only builds a path.
- `"transport": "stdio"` tells the adapter to start the server as a child process and talk over stdin and stdout.

THE CLIENT STARTS THE SERVER. YOU DO NOT.

This is the fact the instructor said took them three days to accept. With stdio, **running the client launches the server**, using exactly the command and args above. So you do not keep the server running in another terminal, as a student pointed out in class, and the separately running server was closed. With HTTP it is the other way round: the server runs on its own, and the client only needs its URL.

Loading the tools

- `MultiServerMCPClient(server_params)` creates the client. Despite the variable being named `mcp_server` in the file, this object is the *client*. Naming it `client` would be clearer.
- `await mcp_server.get_tools()` is where the whole of sections 11 and 12 happens, hidden inside one line: start the process, `initialize`, `initialized`, `tools/list`. As the instructor put it, it can feel like a betrayal after two days of theory, but that is how every framework works. What you get back is a list of LangChain tools with the name, description and schema from the server.
- `get_tools()` is asynchronous, so it needs `await`. `await` is only allowed inside an `async def`, so `main` becomes async, and an async function has to be started with `asyncio.run(main())`. If `async` and `await` are new to you, the course has a recorded video on sync, async and await in the Python gaps section.

Printing `tools` shows a `StructuredTool` with `name='greet'`, `description='This is the tool to greet users'`, an argument schema, and some metadata you can ignore for now. That is the proof that the client sees the server's tool.

The tool calling loop, done by hand

From here it is the same manual loop you wrote in session 5. Only the two lines that load the tools are new.

```
history = [ Human: "Add 4 and 5" ]
            |
            v
llm.invoke(history) -----> AIMessage content=""
                               tool_calls=[ add(num1=4, num2=5) ]
            | append to history
            v
find the tool named "add" in tools
            |
            v
await tool.ainvoke(args) -----> MCP: starts server, tools/call add
                                   result = [{"type":"text", "text":"9", ...}]
            | append ToolMessage(content=result[0]["text"], tool_call_id=...)
            v
await llm.ainvoke(history) -----> AIMessage "4 + 5 = 9"
```

- `bind_tools(tools)` gives the MCP tools to the model, exactly as with local tools. With the tools bound and "hi" as the input, the response had empty content and a `tool_calls` entry for `greet`. The model had decided to call our tool.
- **Why keep a `history` list?** The instructor wondered about this mid-class, then answered it: the second call to the model needs the whole conversation, meaning the question, the model's tool call, and the tool's result.
- `response.tool_calls[0]` takes the first tool call. The code assumes there is one. If the model answers directly without calling a tool, this line raises an `IndexError`.
- `next(t for t in tools if t.name == toolcall["name"])` loops over the tools and returns the first one whose name matches what the model asked for. A generator expression inside `next()` is a short way to find the first match in a list.
- `await tool.ainvoke(toolcall["args"])` runs the tool. It must be `ainvoke`: MCP tools from the adapter are async only, and calling `tool.invoke(...)` raises `NotImplementedError: StructuredTool does not support sync invocation`.
- `result[0]['text']` is needed because the result is a **list of content blocks**, the `content` list from section 4. With the versions used in class it looks like `[{'type': 'text', 'text': '9', 'id': 'lc_...'}]`.
- `ToolMessage(content=..., tool_call_id=toolcall["id"])` sends the result back, linked to the tool call it answers.
- `await llm.ainvoke(history)` is the final call, and the model writes the answer from the tool's result.

`SystemMessage` is imported but no longer used. A system prompt ("You are Telusko, a helpful coding teaching assistant") was in an earlier version and removed at the end.

20 What broke in class, and what it teaches

The instructor chose to build the code live, without rehearsing it, saying that fixing errors together is how real coding goes. Three things went wrong. Each one is a lesson.

1. ImportError from inside langchain_mcp_adapters

The first run of the client failed with an `ImportError`: something could not be imported from `mcp.shared.context`, raised from inside `langchain_mcp_adapters`' callbacks module. Nothing was wrong with the file names or the code. Running `uv add mcp` did not fix it, and neither did a suggestion from the chat to pin `mcp<2`. What worked:

```
uv pip list          # check which mcp version is installed
uv remove mcp
uv add mcp           # fresh install of a compatible version
uv run my_mcp_client.py
```

No code changed. It was only a dependency problem.

THE LESSON

When an import fails *inside* a library, rather than in your own file, suspect a version mismatch between packages before you touch your code. One package expects a newer or older version of another. Check versions with `uv pip list`, compare them with what the library requires, and reinstall or upgrade the one that is out of step.

2. Passing the whole result into ToolMessage

The first version did `ToolMessage(content=result, ...)` and got an error. Then `result.text` gave `'list' object has no attribute 'text'`, and `result.content` did not work either. The fix was to check the type first. It was a list, so the right expression is `result[0]['text']`: first block, then the `text` key.

THE LESSON

When you do not know what an object looks like, do not guess attribute names. Print `type(result)` and `result` first. Thirty seconds of that would have saved the long debugging stretch, which is what the instructor said too.

3. A print inside the tool showed nothing

To confirm the tool was really running, a `print` was added inside it, and nothing appeared. That is when `add` was changed to return 20. The model then answered 9 anyway, because it looked at the tool's 20, knew $4 + 5$ is 9, and corrected it in the final call. But the 20 did show up in the printed tool result, which proved the MCP tool was being called.

The reason the print disappeared is the stdio transport itself. The server's `stdout` is the channel that carries JSON-RPC messages, and the spec says a server must not write anything to stdout that is not a valid MCP message. So ordinary `print()` output cannot just appear in your terminal. We checked this with the same package versions as the class: a `print()` inside a tool vanished silently, while a print to `stderr` showed up in the terminal running the client.

```
import sys

@mcp_server.tool
def add(num1: int, num2: int) -> int:
    """Add two numbers and return the sum."""
    print(f"add called with {num1} and {num2}", file=sys.stderr)    # visible
    return num1 + num2
```

THE LESSON

In a stdio MCP server, never debug with plain `print()`. Write to `sys.stderr`, or use the `logging` module, which writes to stderr by default. The spec explicitly allows servers to write logs to stderr.

One more thing you can see in the terminal

Watch the output when you run the client, and you will see the FastMCP banner more than once. That is not a bug. By default `MultiServerMCPClient` opens a new session for every tool call, so with stdio it starts a fresh server process for `get_tools()` and another for each tool call. When we tested it, calling two tools after `get_tools()` meant three server starts. That is the "currently it is stateless" the instructor mentioned. If you need one session kept open for several calls, the adapter supports that explicitly:

```
from langchain_mcp_adapters.tools import load_mcp_tools

async with client.session("FirstMCPServer") as session:
    tools = await load_mcp_tools(session)
    # every call to these tools reuses this one session
```

21 A cleaner version to try yourself

The class promised to improve these files next week. Until then, here is a tidied version of the same two files, with every issue from section 20 fixed. It is not something shown in class, so treat it as practice. The server:

```
import sys
from fastmcp import FastMCP

mcp_server = FastMCP("FirstMCPServer")

@mcp_server.tool
def greet() -> str:
    """Greet the user with a Telusko welcome message."""
    print("greet was called", file=sys.stderr)
    return "Hello from Telusko, we hope you are fine?"

@mcp_server.tool
def add(num1: int, num2: int) -> int:
    """Add two whole numbers and return their sum."""
    print(f"add was called with {num1} and {num2}", file=sys.stderr)
    return num1 + num2

if __name__ == "__main__":
    mcp_server.run(transport="stdio")
```

The client, handling any number of tool calls, including none:

```

import asyncio
import sys
from pathlib import Path

from dotenv import load_dotenv
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_openai import ChatOpenAI

load_dotenv()

SERVERS = {
    "FirstMCPServer": {
        "command": sys.executable,
        "args": [str(Path(__file__).parent / "my_mcp_server.py")],
        "transport": "stdio",
    }
}

async def main():
    client = MultiServerMCPClient(SERVERS)
    tools = await client.get_tools()
    tools_by_name = {t.name: t for t in tools}

    llm = ChatOpenAI(model="gpt-4o-mini").bind_tools(tools)
    history = [HumanMessage(content="Add 4 and 5, then greet me")]

    response = await llm.ainvoke(history)
    history.append(response)

    for call in response.tool_calls:
        tool = tools_by_name[call["name"]]
        result = await tool.ainvoke(call["args"])
        text = "".join(block["text"] for block in result if block["type"] == "text")
        history.append(ToolMessage(content=text, tool_call_id=call["id"]))

    if response.tool_calls:
        response = await llm.ainvoke(history)

    print(response.content)

```

```
if __name__ == "__main__":
    asyncio.run(main())
```

- A dictionary `tools_by_name` replaces the `next(...)` search.
- The `for` loop handles zero, one or several tool calls, so there is no `IndexError` when the model answers directly.
- Text is joined from every `text` block instead of assuming there is exactly one.
- `ainvoke` is used for both model calls, since we are inside async code anyway.

This handles one round of tool calls, which is all this example needs. If the model might ask for more tools after seeing the results, you loop until there are no tool calls left, or hand the tools to `create_agent`, which runs that loop for you (session 5).

PREVIEW: THE SAME SERVER OVER HTTP

Not covered in class yet, but it shows how little changes. Run the server with

`mcp_server.run(transport="http", host="127.0.0.1", port=8000)` in its own terminal. It then listens at `http://127.0.0.1:8000/mcp`. In the client, replace the command, args and transport entries with `"transport": "http", "url": "http://127.0.0.1:8000/mcp"`. Everything after that stays the same. (Older versions of the adapter call this transport `"streamable_http"`.)

22 Clarifications from the Q&A

- **Do we have to manage the ids ourselves?** No. The SDK creates and matches them. They were explained only so you can read the messages.
- **What if I leave out the JSON-RPC version?** Then it is not a valid JSON-RPC message. In practice you never write it, because the SDK always adds `"2.0"`.
- **Are there headers, like Content-Type?** Over HTTP, yes (JSON content types, and the protocol version is also sent as a header), but the SDK sets them for you.
- **Is JSON-RPC language independent?** Yes. It is just JSON text.
- **Are notifications real time?** They are sent when the change happens, or however the server chooses to send them. Most come from the server, but the client can send them too, as with `notifications/initialized`.
- **Is most of this communication async?** Yes, which is why the client uses `await`, `ainvoke` and `asyncio.run`.
- **Are the connections stateful?** What we built is stateless: every tool call gets a fresh session. The instructor plans to show both approaches later.

- **What about authentication?** It exists in MCP and will be covered later. It does not change the lifecycle you learned today.
- **Does Hugging Face use MCP?** Hugging Face mainly gives you models. MCP works with any model that supports tool calling.
- **How does the client find out the server is down?** The same way as with any backend: errors, timeouts, pings, logging and alerts.
- **Is there a time limit when nothing happens?** You can set timeouts, and retry or give up when they expire. Whether an idle connection gets closed is up to the implementation.
- **Can resources hold an instructions file for the LLM?** Yes, a resource can be any data you choose to provide.
- **Do we need LangChain to create tools on the server?** No. FastMCP has its own decorator.
- **Why not just paste both files into ChatGPT during debugging?** You can, and Claude Code was tried during the session. But the actual fix, reinstalling `mcp`, came from checking versions by hand.

THE ROAD AHEAD, AS LAID OUT AT THE END OF CLASS

Next week continues MCP in code, starting with a review of today's files. After that: RAG (about two or three classes), more MCP, and tool calls. Then a week of building projects with everything learned so far. After that, agents with LangGraph, and then CrewAI. The instructor also said the plan may still change.

23 Key takeaways and official links

- **JSON-RPC 2.0 has three shapes:** a request with `method`, `params` and `id`; a response with `result` or `error` and the same `id`; a notification with a `method` and no `id`.
- **The id matches answers to questions.** A missing id means nobody should reply.
- **List before you use.** `tools/list`, `resources/list` and `prompts/list` are how a client learns what exists.
- **Tool results are a list of content blocks,** which is why the code reads `result[0]['text']`.
- **JSON-RPC suits MCP** because it is lightweight, bidirectional, independent of the transport, and has notifications built in.
- **stdio for local, Streamable HTTP for remote.** With stdio the client launches the server. With HTTP the server runs by itself.
- **Lifecycle as taught: initialize, operate, shut down.** Initialization is three steps, and without step 3 there is no connection.
- **Negotiation covers both version and capabilities,** and both sides must stick to what was agreed.
- **Client capabilities:** roots (where to work), sampling (borrow my LLM), elicitation (ask the user for missing details, never secrets).
- **Server capabilities:** tools, resources, prompts, logging.

- **Shutdown has no message.** Closing the stream or connection is the shutdown.
- **The spec is moving:** the 2026-07-28 revision removes the handshake. Always check which revision a page describes.
- **FastMCP:** `@server.tool`, docstring as description, type hints as schema, `run(transport="stdio")`.
- **langchain-mcp-adapters:** `MultiServerMCPClient` + `await get_tools()` gives you ordinary LangChain tools. Call them with `ainvoke`.
- **Never `print()` to stdout in a stdio server.** Use stderr or logging.
- **An ImportError inside a library usually means mismatched versions,** not a bug in your code.

Official links

What	Link
MCP specification (latest revision)	modelcontextprotocol.io/specification/latest
Lifecycle, as taught (2025-11-25)	modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle
Transports (2025-11-25)	modelcontextprotocol.io/specification/2025-11-25/basic/transports
What changed in 2026-07-28	modelcontextprotocol.io/specification/2026-07-28/changelog
JSON-RPC 2.0 specification	jsonrpc.org/specification
FastMCP documentation	gofastmcp.com
langchain-mcp-adapters	github.com/langchain-ai/langchain-mcp-adapters
LangChain MCP guide	docs.langchain.com/oss/python/langchain/mcp

BEFORE NEXT CLASS

Get today's two files running on your own machine. Then try three things: change `add` back to returning the real sum and add type hints; add a third tool of your own, such as one that returns today's date; and put a stderr print in each tool so you can watch the server being called. If you get an import error, check your package versions first.